

AGEX Core Architecture

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Core의 구조, 책임, 경계, 내부 구성요소, 공통 계약 및 핵심 실행 원칙을 정의한다.

Product Architecture가 AGEX 전체 제품 계층과 도메인 구조를 정의한다면, Core Architecture는 AGEX의 중심 기능을 어떤 내부 모듈과 서비스로 구현할 것인지를 규정한다.

본 문서는 다음 설계의 기준으로 사용한다.

- AGEX Core 범위 정의
- Core Module 구성
- Core Service 책임 분리
- 공통 Resource Model 정의
- Resource Lifecycle 정의
- 실행 요청 처리 구조
- 정책 적용 구조
- 상태 관리 구조
- 내부 Service Interface 정의
- Domain 간 의존성 규칙
- Transaction 및 Consistency 기준
- Event 발행 원칙
- 확장 기능과 Core의 경계
- Core 개발 및 배포 단위
- Core 품질 검증 기준

1.2 적용 범위

본 문서는 AGEX Core에 포함되는 다음 영역을 다룬다.

- Core Resource Management
- Core Identity Context
- Core Tenant Context
- Core Policy Enforcement
- Core Registry
- Core Execution Coordination
- Core Lifecycle Management
- Core State Management
- Core Event Management
- Core Configuration
- Core Audit
- Core Usage Metering
- Core Error Contract
- Core Extension Contract

다음 영역의 상세 설계는 별도 문서에서 정의한다.

- Runtime Architecture
- Agent Framework
- Workflow Engine
- Knowledge Engine
- Memory Engine
- Plugin Framework
- Marketplace
- API Specification
- Security Architecture
- Multi-Tenant Architecture

- Deployment Architecture

1.3 Core의 정의

AGEX Core는 AGEX 제품의 공통 제어 기능과 Resource 관리 기능을 제공하는 핵심 제품 영역이다.

AGEX Core는 개별 Application 기능을 포함하지 않는다.

AGEX Core는 다음 역할을 수행한다.

- AGEX Resource를 정의하고 관리한다.
- Resource 간 관계를 검증한다.
- 실행 요청을 검증하고 Runtime으로 전달한다.
- Tenant, Identity, Permission 및 Policy Context를 적용한다.
- Version과 Lifecycle을 통제한다.
- 실행 상태와 Resource 상태를 관리한다.
- Event, Audit 및 Usage 정보를 기록한다.
- 확장 구성요소가 사용할 공식 계약을 제공한다.

2. Core Architecture 목표

AGEX Core Architecture는 다음 목표를 충족해야 한다.

2.1 공통성

특정 Agent, Workflow, Plugin, Model 또는 Application에 종속되지 않는 공통 기능만 포함해야 한다.

2.2 안정성

확장 기능이나 외부 Provider의 장애가 Core 전체 장애로 이어지지 않아야 한다.

2.3 일관성

모든 AGEX Resource는 동일한 식별, 버전, 상태, 권한 및 감사 규칙을 따라야 한다.

2.4 명시성

모든 입력, 출력, 상태, 오류, 권한 및 의존성은 명시적 계약으로 정의해야 한다.

2.5 추적성

모든 Resource 변경과 실행 요청은 추적 가능해야 한다.

2.6 확장성

새로운 Resource Type과 Provider를 Core 내부 수정 없이 추가할 수 있어야 한다.

2.7 격리성

Tenant, Resource, Execution 및 Extension 간 경계를 명확히 분리해야 한다.

2.8 교체 가능성

Storage, Queue, Search, Cache 및 Provider는 Adapter를 통해 교체할 수 있어야 한다.

2.9 운영 가능성

Core의 상태, 부하, 오류, 비용 및 변경 이력을 운영자가 확인할 수 있어야 한다.

2.10 보안성

모든 Core 동작은 인증, 권한, 정책 및 감사 기준을 따라야 한다.

3. Core Architecture 원칙

AGEX Core는 다음 원칙을 따른다.

- Domain Separation
- Single Responsibility
- Explicit Contract
- Versioned Resource
- Immutable Published Version
- Policy Before Execution
- Tenant Context Mandatory
- Event After State Change
- Audit for Sensitive Action
- Idempotent Command

- Durable State
 - Stateless Service Where Possible
 - Provider Abstraction
 - Extension Isolation
 - Fail-Safe Default
 - Least Privilege
 - Schema-First
 - Backward Compatibility
 - Observable by Default
 - No Direct Cross-Domain Database Access
-

4. Core 전체 구조

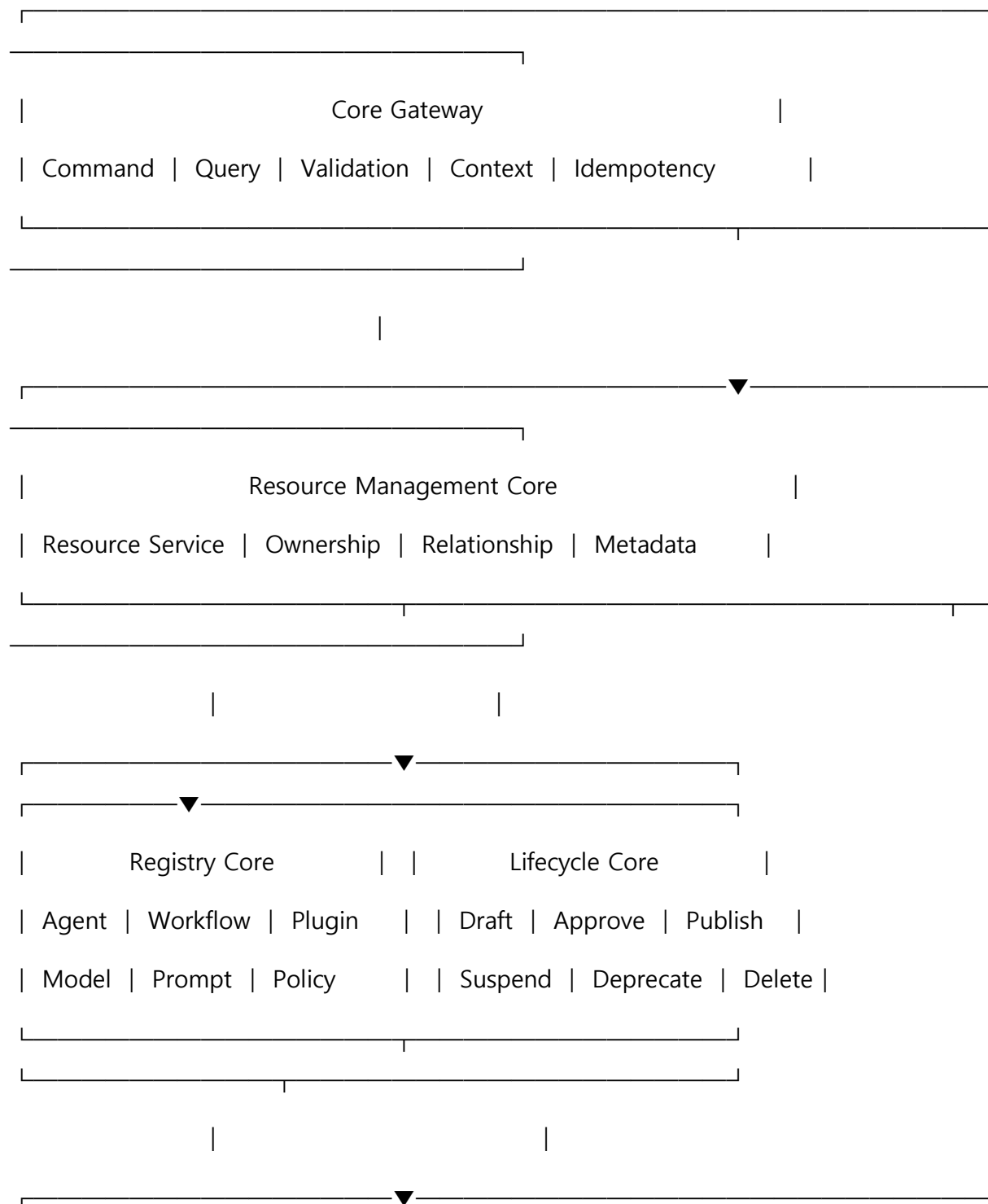
AGEX Core는 다음 주요 영역으로 구성한다.

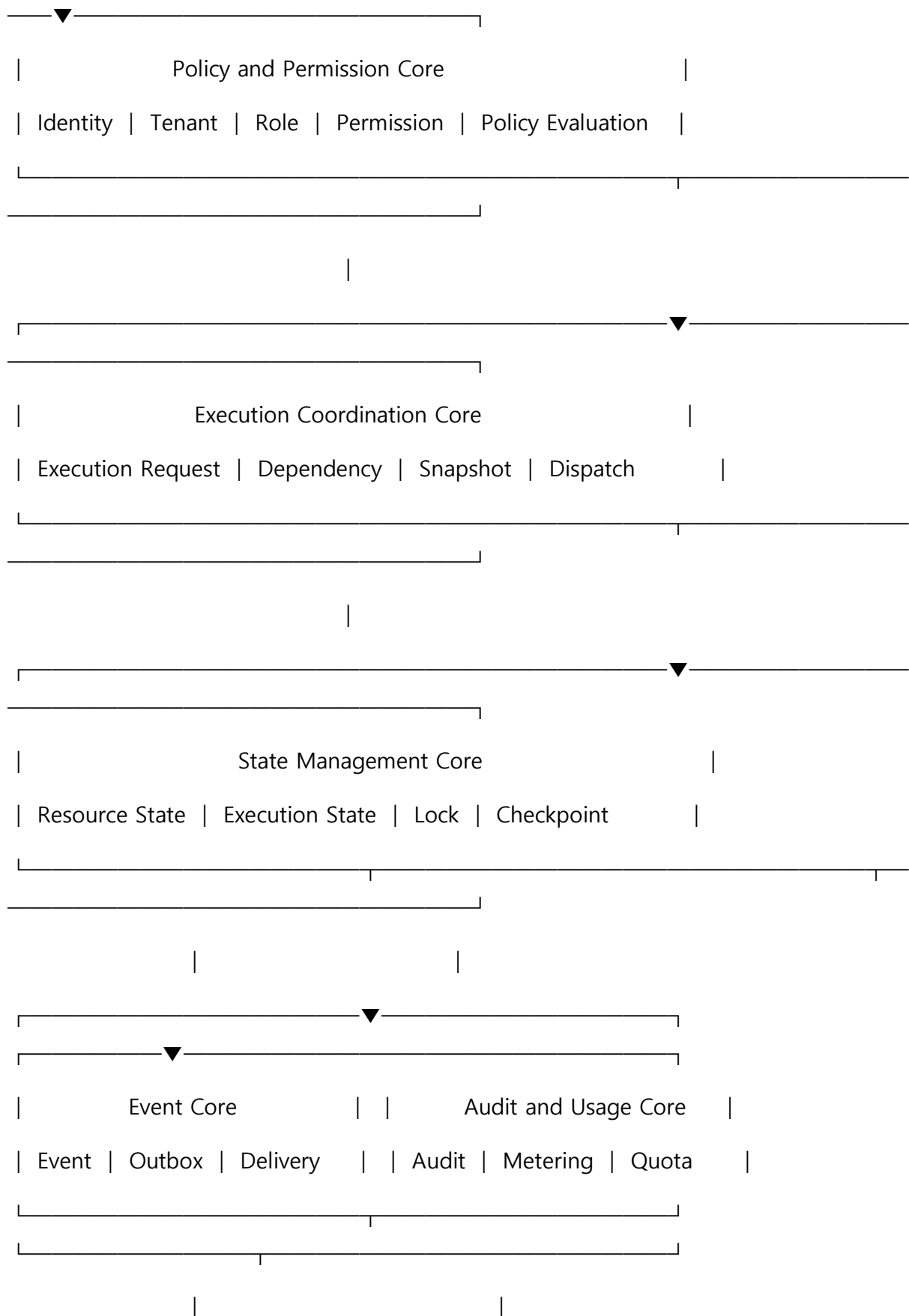
1. Core Gateway
2. Resource Management Core
3. Registry Core
4. Lifecycle Core
5. Policy and Permission Core
6. Execution Coordination Core
7. State Management Core
8. Event Core
9. Configuration Core
10. Audit Core
11. Usage and Quota Core
12. Schema and Contract Core

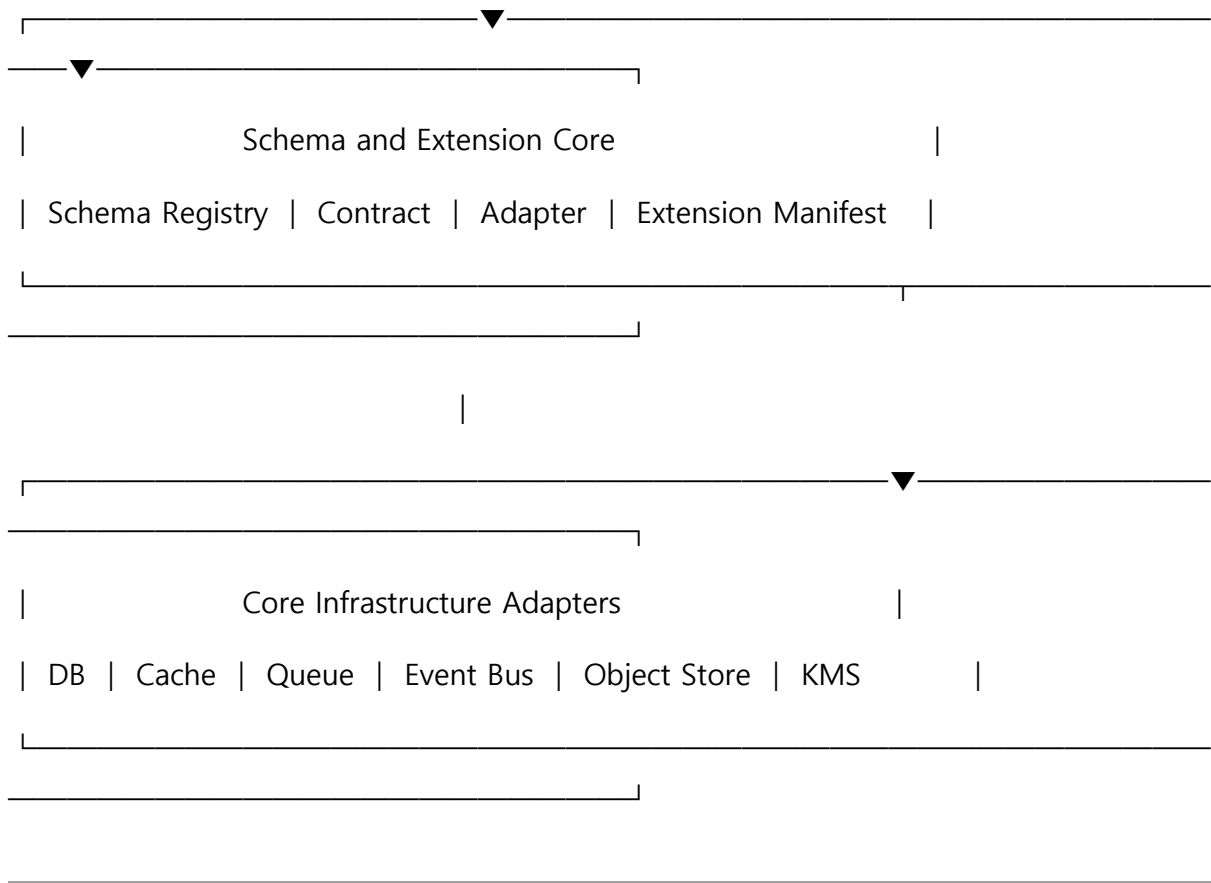
13. Extension Core

14. Core Infrastructure Adapters

5. Core 개념 구조







6. Core Gateway

6.1 정의

Core Gateway는 AGEX Core에 전달되는 모든 Command와 Query의 내부 진입점이다.

외부 API Gateway와 구분되며, 외부 요청을 Core Domain 명령으로 변환하는 역할을 수행한다.

6.2 주요 책임

Core Gateway는 다음을 수행한다.

- Command 수신
- Query 수신
- 요청 형식 검증
- Tenant Context 확인
- Identity Context 확인
- Permission Context 확인

- Request ID 생성
- Correlation ID 연결
- Idempotency Key 확인
- Command Handler 선택
- Query Handler 선택
- 오류 표준화
- 응답 형식 표준화

6.3 Command와 Query 분리

Core는 상태를 변경하는 Command와 상태를 조회하는 Query를 구분한다.

6.3.1 Command

Command는 Resource나 Execution 상태를 변경한다.

예시는 다음과 같다.

- CreateResource
- UpdateResource
- PublishResource
- SuspendResource
- StartExecution
- CancelExecution
- InstallExtension
- ChangePolicy

6.3.2 Query

Query는 상태를 변경하지 않고 데이터를 조회한다.

예시는 다음과 같다.

- GetResource
- ListResources

- GetExecution
- GetDependencyGraph
- GetAuditRecords
- GetUsageSummary

6.4 Command 처리 원칙

모든 Command는 다음 정보를 포함해야 한다.

- Command ID
- Command Type
- Tenant ID
- Principal ID
- Resource Type
- Resource ID
- Requested At
- Idempotency Key
- Payload
- Trace Context
- Security Context

6.5 Query 처리 원칙

모든 Query는 다음을 포함해야 한다.

- Query Type
- Tenant ID
- Principal ID
- Filter
- Sort
- Pagination

- Projection
 - Trace Context
-

7. Core Resource Model

7.1 정의

Core Resource Model은 AGEX의 모든 주요 구성요소에 공통으로 적용되는 기본 데이터 구조이다.

7.2 Core Resource 공통 속성

모든 Core Resource는 최소한 다음 속성을 가져야 한다.

- Resource ID
- Resource Type
- Tenant ID
- Namespace
- Name
- Display Name
- Description
- Version
- Revision
- Lifecycle State
- Owner ID
- Created By
- Created At
- Updated By
- Updated At
- Labels

- Tags
- Metadata
- Specification
- Status
- Permission Reference
- Policy Reference
- Compatibility
- Checksum

7.3 Specification과 Status 분리

Resource는 Specification과 Status를 분리한다.

7.3.1 Specification

사용자 또는 관리자가 원하는 Resource 정의 상태이다.

7.3.2 Status

시스템이 실제로 확인한 현재 상태이다.

예를 들어 Plugin의 Specification은 설치 요청 상태를 나타내고, Status는 실제 설치 완료 여부를 나타낼 수 있다.

7.4 Version과 Revision

Version과 Revision은 구분한다.

- Version은 외부에 공개되는 기능 단위 변경이다.
- Revision은 동일 Version 내부의 편집 이력이다.

Published 상태의 Version은 불변으로 취급한다.

수정이 필요한 경우 새로운 Version을 생성한다.

7.5 Resource 식별

Resource ID는 전역적으로 충돌하지 않아야 한다.

표시 이름은 중복될 수 있으나 Namespace와 Tenant 범위 안에서 식별 가능해야 한다.

7.6 Resource Reference

다른 Resource를 참조할 때는 최소한 다음을 포함해야 한다.

- Resource Type
 - Resource ID
 - Version Constraint
 - Tenant Scope
 - Required or Optional
 - Compatibility Rule
-

8. Resource Management Core

8.1 정의

Resource Management Core는 AGEX Resource의 생성, 조회, 수정, 복제, 삭제 및 관계 관리를 담당한다.

8.2 주요 기능

- Resource 생성
- Resource 조회
- Resource 수정
- Resource 복제
- Resource 비교
- Resource Import
- Resource Export
- Resource 관계 설정
- Ownership 변경
- Namespace 변경
- Label 및 Metadata 관리

- Soft Delete
- Restore
- Permanent Delete 요청

8.3 Resource 생성 절차

Resource 생성은 다음 순서로 처리한다.

1. 요청 Schema 검증
2. Tenant 확인
3. Principal 확인
4. 생성 권한 확인
5. Namespace 확인
6. 이름 중복 정책 확인
7. Specification 검증
8. Policy 적용
9. Resource ID 생성
10. 초기 Version 생성
11. 초기 Lifecycle State 설정
12. 저장
13. Audit 기록
14. ResourceCreated Event 발행

8.4 Resource 수정 절차

Resource 수정 시 다음을 확인한다.

- 수정 가능한 Lifecycle State인지
- 수정 권한이 있는지
- Published Version인지
- 동시 수정 충돌이 있는지

- 의존 Resource에 미치는 영향이 있는지
- Schema가 유효한지
- 새로운 Revision이 필요한지
- 새로운 Version이 필요한지

8.5 Optimistic Lock

동시 수정 충돌을 방지하기 위해 Revision 또는 ETag 기반 Optimistic Lock을 사용한다. 수정 요청은 현재 Revision 값을 포함해야 한다.

8.6 삭제 정책

Resource 삭제는 다음 단계로 구분한다.

1. Soft Delete
2. Retention
3. Dependency Check
4. Approval
5. Permanent Delete
6. Deletion Audit
7. Tombstone 유지

참조 중인 Resource는 즉시 영구 삭제할 수 없다.

9. Registry Core

9.1 정의

Registry Core는 AGEX에서 사용 가능한 Resource 정의와 버전을 등록하고 조회하는 공통 등록 체계이다.

9.2 Registry 유형

- Agent Registry
- Workflow Registry

- Plugin Registry
- Model Registry
- Prompt Registry
- Policy Registry
- Schema Registry
- Knowledge Package Registry
- Extension Registry
- Runtime Image Registry

9.3 Registry Entry

Registry Entry는 다음 정보를 포함한다.

- Registry Entry ID
- Resource Reference
- Resource Version
- Publication State
- Compatibility
- Dependencies
- Capabilities
- Checksum
- Signature
- Publisher
- Published At
- Deprecated At
- Supported Runtime Versions

9.4 Registry 조회

Registry는 다음 조건으로 조회할 수 있어야 한다.

- Resource Type
- Capability
- Version
- Lifecycle State
- Tenant
- Publisher
- Compatibility
- Label
- Trust Level

9.5 불변성

Published Registry Entry는 수정할 수 없다.

변경이 필요한 경우 새로운 Version을 등록한다.

9.6 Registry 무결성

Registry에 등록되는 모든 Package와 Manifest는 다음을 검증한다.

- Schema 유효성
- Checksum
- Signature
- Dependency
- Version Compatibility
- Permission Declaration
- Security Scan 결과
- Publisher Identity

10. Lifecycle Core

10.1 정의

Lifecycle Core는 AGEX Resource의 상태 전이와 승인 절차를 관리한다.

10.2 기본 Lifecycle

Resource의 기본 Lifecycle은 다음과 같다.

Draft

→ Validating

→ Review

→ Approved

→ Published

→ Active

→ Suspended

→ Deprecated

→ Archived

→ Deleted

모든 Resource가 모든 상태를 사용해야 하는 것은 아니지만, 상태 전이 규칙은 명시적으로 정의해야 한다.

10.3 상태 전이 규칙

각 상태 전이는 다음 정보를 가진다.

- From State
- To State
- Required Permission
- Required Validation
- Required Approval
- Side Effect
- Event
- Audit Requirement

- Rollback Rule

10.4 Published 상태

Published 상태의 Resource는 다음 조건을 충족해야 한다.

- Schema 검증 완료
- Dependency 검증 완료
- Policy 검증 완료
- Security 검증 완료
- Compatibility 검증 완료
- 필수 승인 완료
- Checksum 생성 완료
- Registry 등록 완료

10.5 Suspended 상태

Suspended Resource는 신규 실행에 사용할 수 없다.

기존 실행에 대한 처리 정책은 Resource 유형별로 정의한다.

10.6 Deprecated 상태

Deprecated Resource는 신규 사용을 제한하거나 경고를 발생시킨다.

대체 Version 정보를 제공해야 한다.

10.7 Archived 상태

Archived Resource는 실행할 수 없으며 조회와 감사 목적의 보존만 허용한다.

11. Policy and Permission Core

11.1 정의

Policy and Permission Core는 Resource 접근과 실행 요청의 허용 여부를 결정한다.

11.2 주요 구성요소

- Identity Resolver

- Tenant Resolver
- Role Resolver
- Permission Resolver
- Resource Scope Resolver
- Policy Decision Point
- Policy Enforcement Point
- Policy Information Point
- Policy Administration Point

11.3 권한 모델

AGEX는 다음 요소를 조합한 권한 모델을 사용한다.

- Role-Based Access Control
- Attribute-Based Access Control
- Resource-Based Access Control
- Policy-Based Access Control
- Tenant Scope
- Environment Scope
- Time Constraint
- Data Classification
- Risk Level

11.4 Permission 구조

Permission은 다음 형식을 따른다.

Action + Resource Type + Resource Scope + Condition

예시는 다음 구조를 가진다.

execute + agent + tenant namespace + approved version only

11.5 Policy 평가 입력

Policy Engine은 다음 정보를 입력으로 사용한다.

- Principal
- Tenant
- Role
- Resource
- Action
- Environment
- Time
- Device
- Network
- Risk Level
- Data Classification
- Cost Estimate
- Approval State
- Execution Context

11.6 Policy 결과

Policy 평가 결과는 다음 중 하나이다.

- Permit
- Deny
- Permit With Obligation
- Require Approval
- Require Additional Authentication
- Indeterminate

11.7 Obligation

Permit With Obligation은 실행 전에 추가 조건을 적용한다.

예시는 다음과 같다.

- 출력 마스킹
- 추가 Audit
- 실행 시간 제한
- 비용 제한
- 특정 Runtime 사용
- Human Approval
- 결과 보존 제한

11.8 기본 거부

명시적으로 허용되지 않은 요청은 거부한다.

Policy 평가 실패나 오류도 기본적으로 거부로 처리한다.

12. Execution Coordination Core

12.1 정의

Execution Coordination Core는 검증된 실행 요청을 Runtime에 전달하기 전에 실행에 필요한 모든 구성과 의존성을 확정한다.

12.2 주요 책임

- Execution Request 생성
- Resource Version 확정
- Dependency Resolution
- Policy 평가
- Runtime 선택
- Execution Snapshot 생성
- Budget 확인
- Quota 확인

- Execution Queue 전달
- Execution ID 발급
- 초기 상태 저장

12.3 Execution Request

Execution Request는 다음 정보를 포함한다.

- Execution ID
- Tenant ID
- Principal ID
- Execution Type
- Target Resource
- Target Version
- Input
- Context
- Permission Context
- Policy Context
- Budget
- Deadline
- Priority
- Environment
- Trace Context
- Parent Execution ID
- Correlation ID

12.4 Dependency Resolution

실행 전에 다음 의존성을 해석한다.

- Agent Version

- Workflow Version
- Plugin Version
- Model Configuration
- Prompt Version
- Policy Version
- Knowledge Scope
- Memory Policy
- Schema Version
- Runtime Version

12.5 Execution Snapshot

실행 시작 시점의 모든 구성은 Snapshot으로 고정한다.

Snapshot은 다음 정보를 포함한다.

- Resource Definition
- Dependency Versions
- Policy Decisions
- Permission Scope
- Model Policy
- Plugin Access
- Knowledge Access
- Memory Policy
- Cost Limit
- Runtime Configuration
- Environment Configuration

실행 중 원본 Resource가 변경되어도 기존 실행의 Snapshot에는 영향을 주지 않는다.

12.6 실행 승인

승인이 필요한 실행은 Runtime에 전달하지 않고 Approval Pending 상태로 저장한다.
승인 완료 후 동일한 Execution Request를 다시 검증한다.

13. State Management Core

13.1 정의

State Management Core는 Resource 상태와 Execution 상태를 지속적으로 저장하고 일관성을 유지한다.

13.2 상태 유형

- Resource State
- Lifecycle State
- Execution State
- Workflow State
- Agent State
- Approval State
- Installation State
- Deployment State
- Synchronization State

13.3 상태 전이

상태 전이는 허용된 State Machine을 통해서만 수행한다.

데이터베이스 값을 직접 수정하여 상태를 변경해서는 안 된다.

13.4 상태 변경 기록

모든 상태 변경은 다음 정보를 기록한다.

- Previous State
- New State
- Trigger

- Principal
- Timestamp
- Reason
- Related Event
- Related Execution
- Revision

13.5 Checkpoint

장기 실행은 Checkpoint를 저장해야 한다.

Checkpoint는 다음을 포함한다.

- Current Step
- Completed Steps
- Pending Steps
- Intermediate Output
- Variables
- Retry Count
- Error State
- External Reference
- Updated At

13.6 Lock

중복 처리 방지를 위해 필요한 Resource와 Execution에 Lock을 적용할 수 있다.

Lock은 영구적이어서는 안 되며 만료 시간을 가져야 한다.

13.7 상태 복구

시스템 재시작 후 다음 상태를 복구할 수 있어야 한다.

- Queued
- Running

- Waiting
 - Retrying
 - Approval Pending
 - Paused
 - Compensating
-

14. Event Core

14.1 정의

Event Core는 AGEX 내부의 상태 변화와 실행 결과를 Event로 발행하고 전달한다.

14.2 Event 유형

- Domain Event
- Integration Event
- Audit Event
- Execution Event
- Lifecycle Event
- Security Event
- Usage Event
- System Event

14.3 Domain Event

Domain Event는 Core 내부의 상태 변화를 나타낸다.

예시는 다음과 같다.

- ResourceCreated
- ResourceUpdated
- ResourcePublished
- ResourceSuspended

- ExecutionCreated
- ExecutionStarted
- ExecutionCompleted
- ExecutionFailed
- PolicyDenied
- QuotaExceeded

14.4 Event 구조

모든 Event는 다음 필드를 포함한다.

- Event ID
- Event Type
- Event Version
- Tenant ID
- Source
- Subject
- Occurred At
- Published At
- Correlation ID
- Causation ID
- Principal ID
- Trace Context
- Payload
- Security Classification

14.5 Transactional Outbox

상태 변경과 Event 발행의 불일치를 방지하기 위해 Transactional Outbox Pattern을 사용한다.

상태 저장과 Outbox 기록은 동일 Transaction에서 수행한다.

14.6 중복 처리

Event Consumer는 중복 Event를 처리할 수 있어야 한다.

Event ID 기반 Idempotency를 적용한다.

14.7 Event 순서

모든 Event에 전역 순서를 보장하지 않는다.

순서가 필요한 경우 Aggregate 또는 Execution 단위 Sequence를 사용한다.

14.8 Dead Letter

전달에 반복 실패한 Event는 Dead Letter Queue로 이동한다.

운영자는 원인 확인 후 재처리할 수 있어야 한다.

15. Configuration Core

15.1 정의

Configuration Core는 Platform, Environment, Tenant 및 Resource 수준의 설정을 관리한다.

15.2 Configuration 계층

Configuration은 다음 계층으로 구분한다.

1. Platform Default
2. Environment Configuration
3. Tenant Configuration
4. Namespace Configuration
5. Resource Configuration
6. Execution Override

15.3 적용 우선순위

하위 계층 설정이 상위 계층 설정을 덮어쓸 수 있으나, 보안과 정책의 최소 기준은 완화할 수 없다.

15.4 Configuration 속성

Configuration은 다음 정보를 포함한다.

- Configuration ID
- Scope
- Key
- Value
- Value Type
- Version
- Effective From
- Effective Until
- Secret 여부
- Mutable 여부
- Restart Required 여부
- Owner
- Audit Requirement

15.5 동적 설정

동적 변경 가능한 설정과 재시작이 필요한 설정을 구분한다.

15.6 Configuration Snapshot

Execution은 실행 시작 시점의 Configuration Snapshot을 사용한다.

16. Audit Core

16.1 정의

Audit Core는 AGEX의 중요 관리 행위와 실행 행위를 변경 불가능한 기록으로 저장한다.

16.2 감사 대상

다음 행위는 반드시 Audit 대상이다.

- 로그인 및 인증 실패
- Role 및 Permission 변경
- Policy 변경
- Resource 생성, 수정, 배포, 폐기
- Agent 실행
- Workflow 실행
- Plugin 설치 및 실행
- Secret 접근
- Knowledge 접근
- Memory 접근 및 삭제
- Tenant 설정 변경
- 비용 한도 변경
- 관리자 강제 종료
- Marketplace 배포
- 데이터 Export 및 Delete

16.3 Audit Record

Audit Record는 다음 정보를 포함한다.

- Audit ID
- Tenant ID
- Principal ID
- Action
- Resource
- Timestamp
- Source IP
- Device Context

- Request ID
- Correlation ID
- Previous Value
- New Value
- Result
- Failure Reason
- Policy Decision
- Risk Level

16.4 불변성

Audit Record는 수정하거나 삭제할 수 없다.

보존 기간 종료 시 정책에 따라 별도 폐기 절차를 적용한다.

16.5 감사 무결성

Audit Log는 Hash Chain, Signature 또는 외부 불변 저장소를 통해 무결성을 검증할 수 있어야 한다.

17. Usage and Quota Core

17.1 정의

Usage and Quota Core는 AGEX Resource와 실행 사용량을 측정하고 제한한다.

17.2 측정 대상

- API Request
- Agent Execution
- Workflow Execution
- Model Invocation
- Token Usage
- Plugin Invocation

- Compute Time
- Storage
- Network
- Knowledge Indexing
- Memory Storage
- Event Volume
- Concurrent Execution

17.3 Usage Record

Usage Record는 다음 정보를 포함한다.

- Usage ID
- Tenant ID
- Principal ID
- Resource Type
- Resource ID
- Execution ID
- Usage Type
- Quantity
- Unit
- Unit Cost
- Total Cost
- Measured At
- Billing Period
- Metadata

17.4 Quota 유형

- Hard Quota

- Soft Quota
- Rate Quota
- Concurrent Quota
- Budget Quota
- Storage Quota
- Time-Based Quota

17.5 Quota 처리

Hard Quota 초과 요청은 거부한다.

Soft Quota 초과 시 경고하거나 승인 절차로 전환한다.

17.6 사전 비용 검증

실행 전에 예상 비용이 계산 가능한 경우 Budget Policy를 평가한다.

예상 비용이 한도를 초과하면 실행을 거부하거나 승인을 요구한다.

18. Schema and Contract Core

18.1 정의

Schema and Contract Core는 AGEX 구성요소 간 데이터 계약을 정의하고 검증한다.

18.2 Schema 대상

- API Input
- API Output
- Event Payload
- Agent Input
- Agent Output
- Workflow Input
- Workflow Output
- Plugin Input

- Plugin Output
- Model Structured Output
- Configuration
- Manifest
- Package Metadata

18.3 Schema Registry

Schema Registry는 다음 정보를 관리한다.

- Schema ID
- Schema Name
- Version
- Format
- Owner
- Compatibility Mode
- Status
- Checksum
- Created At
- Deprecated At

18.4 Compatibility Mode

Schema는 다음 호환성 정책을 선택할 수 있다.

- Backward Compatible
- Forward Compatible
- Full Compatible
- Breaking Change Allowed
- No Compatibility Guarantee

18.5 계약 검증 시점

Schema 검증은 다음 시점에 수행한다.

- Resource 등록
- Resource 배포
- API 요청
- Event 발행
- Event 소비
- Plugin 실행
- Model 출력
- Workflow Step 연결
- Package 설치

18.6 Breaking Change

Breaking Change는 Major Version 증가를 요구한다.

기존 Version은 지원 정책에 따라 유지한다.

19. Extension Core

19.1 정의

Extension Core는 AGEX Core 외부의 확장 기능이 안전하게 등록되고 실행되도록 하는 공통 계약을 제공한다.

19.2 Extension 유형

- Plugin
- Model Adapter
- Knowledge Connector
- Memory Provider
- Policy Module
- Event Handler

- Runtime Adapter
- Storage Adapter
- UI Extension
- SDK Extension

19.3 Extension Manifest

모든 Extension은 다음 정보를 선언해야 한다.

- Extension ID
- Name
- Version
- Type
- Provider
- Capabilities
- Input Schema
- Output Schema
- Required Permissions
- Required Secrets
- Network Access
- Resource Requirements
- Compatibility
- Dependencies
- Health Check
- Lifecycle Hooks
- Signature
- Checksum

19.4 Extension 실행 제한

Extension은 다음 제한을 받는다.

- Tenant Scope
- Permission Scope
- CPU Limit
- Memory Limit
- Execution Timeout
- Network Policy
- File System Policy
- Secret Scope
- API Scope
- Concurrent Limit

19.5 Core 접근 제한

Extension은 Core Database에 직접 접근할 수 없다.

공식 API, Event, SDK 또는 Adapter Interface를 통해서만 Core 기능을 사용한다.

19.6 Extension 장애 격리

Extension 장애는 다음 방식으로 격리한다.

- Process Isolation
 - Container Isolation
 - Timeout
 - Circuit Breaker
 - Rate Limit
 - Health Check
 - Automatic Disable
 - Failure Threshold
-

20. Core Infrastructure Adapters

20.1 정의

Core Infrastructure Adapters는 Core Domain과 실제 인프라 구현을 분리한다.

20.2 Adapter 유형

- Repository Adapter
- Cache Adapter
- Queue Adapter
- Event Bus Adapter
- Object Storage Adapter
- Secret Adapter
- KMS Adapter
- Search Adapter
- Lock Adapter
- Clock Adapter
- Notification Adapter

20.3 Repository Interface

각 Domain은 자체 Repository Interface를 정의한다.

다른 Domain의 Repository를 직접 호출할 수 없다.

20.4 Adapter 요구사항

Adapter는 다음 기능을 제공해야 한다.

- Configuration Validation
- Connection Management
- Health Check
- Error Mapping
- Retry Support

- Metric Exposure
- Transaction Support 여부
- Consistency Model
- Version Compatibility

20.5 기술 종속성 제한

Domain Code는 특정 Database, Queue 또는 Cloud SDK를 직접 Import하지 않는다.

21. Domain 경계

21.1 Domain 분리

Core는 다음 주요 Domain으로 구분한다.

- Resource Domain
- Registry Domain
- Lifecycle Domain
- Identity Domain
- Tenant Domain
- Policy Domain
- Execution Domain
- State Domain
- Event Domain
- Configuration Domain
- Audit Domain
- Usage Domain
- Schema Domain
- Extension Domain

21.2 Domain 간 통신

Domain 간 통신은 다음 방식 중 하나를 사용한다.

- Application Service Call
- Domain Service Interface
- Event
- Command
- Query
- Published Contract

21.3 직접 데이터 접근 금지

한 Domain이 다른 Domain의 Database Table이나 Repository에 직접 접근해서는 안 된다.

21.4 순환 의존성 금지

Domain 간 순환 의존성을 허용하지 않는다.

필요한 경우 Event 또는 상위 Orchestration Service로 분리한다.

21.5 Shared Kernel

공유 가능한 최소 요소만 Shared Kernel에 포함한다.

- Identifier
- Timestamp
- Tenant Context
- Principal Context
- Error Contract
- Pagination
- Trace Context
- Basic Metadata

Business Rule이나 Domain Logic은 Shared Kernel에 포함하지 않는다.

22. Core Service 구성

22.1 초기 구현 전략

초기 구현은 Modular Monolith 또는 명확한 Module Boundary를 가진 Service 구조로 시작할 수 있다.

단, 각 Module은 향후 독립 Service로 분리할 수 있도록 다음을 보장해야 한다.

- 독립 Interface
- 독립 Repository
- 독립 Schema
- 독립 Event
- 명확한 책임
- 직접 Table 접근 금지

22.2 독립 Service 분리 기준

다음 조건이 충족되면 독립 Service로 분리할 수 있다.

- 부하 특성이 크게 다름
- 독립 확장이 필요함
- 장애 격리가 필요함
- 보안 경계가 다름
- 배포 주기가 다름
- 데이터 소유권이 명확함
- 팀 소유권이 분리됨

22.3 권장 Core Service

장기적으로 다음 Service 구성을 권장한다.

- Resource Service
- Registry Service
- Lifecycle Service

- Identity Service
 - Tenant Service
 - Policy Service
 - Execution Coordination Service
 - State Service
 - Event Service
 - Configuration Service
 - Audit Service
 - Usage Service
 - Schema Service
 - Extension Service
-

23. Transaction 및 Consistency

23.1 Transaction 경계

단일 Domain 내부의 상태 변경은 로컬 Transaction으로 처리한다.

여러 Domain에 걸친 변경은 분산 Transaction 대신 Event 기반 조정을 사용한다.

23.2 Strong Consistency 대상

다음 영역은 강한 일관성이 필요하다.

- Permission 변경
- Policy 상태
- Resource Version 생성
- Lifecycle 상태 전이
- Execution 상태 전이
- Quota 차감
- Secret Reference

- Audit 생성 요청

23.3 Eventual Consistency 대상

다음 영역은 최종 일관성을 허용할 수 있다.

- Search Index
- Usage Aggregation
- Dashboard Metric
- Marketplace 통계
- Cache
- Notification
- Secondary Read Model

23.4 Saga

여러 Domain에 걸친 장기 처리에는 Saga Pattern을 사용할 수 있다.

각 단계는 보상 작업을 정의해야 한다.

23.5 Idempotency

다음 작업은 Idempotent하게 처리해야 한다.

- Resource 생성
- Execution 시작
- Event 소비
- Plugin 설치
- Usage 기록
- Approval 처리
- Callback 처리

24. Core Read Model

24.1 정의

조회 성능과 운영 편의를 위해 Write Model과 별도의 Read Model을 구성할 수 있다.

24.2 Read Model 대상

- Resource 목록
- Execution Dashboard
- Dependency Graph
- Audit Search
- Usage Summary
- Tenant Overview
- Registry Catalog
- Lifecycle History

24.3 갱신 방식

Read Model은 Domain Event를 구독하여 갱신한다.

24.4 Source of Truth

Read Model은 Source of Truth가 아니다.

정합성 문제가 발생하면 Domain Store 기준으로 재구성할 수 있어야 한다.

25. Core 오류 모델

25.1 오류 범주

Core 오류는 다음으로 분류한다.

- Validation Error
- Authentication Error
- Authorization Error
- Policy Denial
- Resource Not Found
- Resource Conflict

- Version Conflict
- Invalid Lifecycle Transition
- Dependency Error
- Quota Exceeded
- Execution Rejected
- Extension Error
- Infrastructure Error
- Internal Error

25.2 오류 구조

모든 Core 오류는 다음 구조를 따른다.

- Error Code
- Error Category
- Message
- Retryable
- Tenant ID
- Resource Reference
- Execution ID
- Correlation ID
- Details
- Suggested Action
- Timestamp

25.3 오류 노출

내부 구현 정보, Secret, Stack Trace 및 민감한 데이터는 외부 응답에 포함하지 않는다.

25.4 Retryable 구분

오류는 재시도 가능 여부를 명시해야 한다.

재시도 불가능한 오류를 반복 실행하지 않는다.

26. Core 관측성

26.1 Log

모든 Core Service는 구조화된 Log를 생성한다.

필수 필드는 다음과 같다.

- Timestamp
- Service
- Environment
- Tenant ID
- Principal ID
- Request ID
- Correlation ID
- Trace ID
- Resource ID
- Execution ID
- Level
- Message
- Error Code

26.2 Metric

Core는 다음 Metric을 제공해야 한다.

- Request Count
- Error Rate
- Latency
- Command 처리량

- Query 처리량
- Event 처리량
- Queue Depth
- Lock 충돌
- Transaction 실패
- Policy 거부
- Quota 초과
- Resource 생성량
- Execution 생성량

26.3 Trace

Command와 Query는 전체 처리 경로에서 Trace를 유지한다.

26.4 Health Check

각 Service는 다음 Health 상태를 제공한다.

- Liveness
- Readiness
- Dependency Health
- Degraded State

27. Core 보안 경계

27.1 신뢰 경계

Core는 다음 경계를 별도로 취급한다.

- External Client Boundary
- Application Layer Boundary
- Extension Boundary
- Runtime Boundary

- Tenant Boundary
- Infrastructure Boundary
- Administration Boundary

27.2 Service Identity

모든 Core Service는 자체 Service Identity를 가진다.

Service 간 요청도 인증과 권한 검증을 수행한다.

27.3 Secret 접근

Core Service는 필요한 Secret만 접근할 수 있다.

Secret 값은 Log, Event 또는 일반 Configuration에 포함하지 않는다.

27.4 관리자 기능

관리자 기능도 일반 Policy 평가와 Audit를 거쳐야 한다.

관리자라는 이유만으로 모든 권한을 자동 부여하지 않는다.

27.5 내부 요청 검증

내부 Service 요청도 다음을 검증한다.

- Service Identity
- Tenant Context
- Action
- Resource Scope
- Policy
- Trace Context

28. Core 배포 구조 원칙

28.1 Stateless 우선

API 및 Command 처리 Service는 가능한 한 Stateless하게 구현한다.

상태는 외부 저장소에 보관한다.

28.2 Horizontal Scaling

Core Service는 수평 확장할 수 있어야 한다.

28.3 Rolling Deployment

새 Version 배포 시 기존 요청을 중단하지 않는 Rolling Deployment를 지원해야 한다.

28.4 Schema Migration

Database Schema 변경은 이전 Version과의 호환성을 고려하여 단계적으로 수행한다.

28.5 Feature Flag

위험한 기능이나 점진적 배포가 필요한 기능은 Feature Flag를 사용할 수 있다.

Feature Flag는 Tenant와 Environment 범위로 제어할 수 있어야 한다.

29. Core 테스트 기준

29.1 Unit Test

각 Domain Rule과 State Transition을 검증한다.

29.2 Contract Test

API, Event, Schema 및 Adapter 계약을 검증한다.

29.3 Integration Test

Repository, Queue, Event Bus 및 외부 Adapter 연계를 검증한다.

29.4 Tenant Isolation Test

Tenant 간 Resource와 실행이 완전히 분리되는지 검증한다.

29.5 Policy Test

허용, 거부, 승인 요구 및 Obligation 적용을 검증한다.

29.6 Lifecycle Test

모든 허용 및 금지 상태 전이를 검증한다.

29.7 Idempotency Test

중복 Command와 Event가 중복 처리되지 않는지 검증한다.

29.8 Failure Test

Database, Queue, Event Bus 및 Adapter 장애 상황을 검증한다.

29.9 Recovery Test

재시작 후 상태와 대기 작업이 정상 복구되는지 검증한다.

29.10 Load Test

Resource 조회, 실행 요청, Event 처리 및 Audit 기록 부하를 검증한다.

30. Core 성능 기준

30.1 API 처리

단순 Resource 조회와 Command 검증은 장기 실행 Task와 분리해야 한다.

30.2 비동기 처리

다음 작업은 비동기 처리한다.

- 대규모 Import
- 대규모 Export
- Dependency Scan
- Security Scan
- Package Validation
- Indexing
- 장기 실행
- 대량 Event 재처리

30.3 Cache 적용

Cache는 다음에 사용할 수 있다.

- Resource Metadata
- Permission Decision
- Policy Bundle

- Registry Lookup
- Schema Lookup
- Configuration

단, Cache 만료와 무효화 정책을 명확히 정의해야 한다.

30.4 Hot Path

실행 요청의 Hot Path에는 불필요한 외부 호출을 포함하지 않는다.

필요한 정책과 Configuration은 Snapshot 또는 Cache를 사용할 수 있다.

31. Core 확장 기준

새로운 기능을 Core에 포함하려면 다음 조건을 모두 검토해야 한다.

1. 특정 Application에 종속되지 않는가.
2. 두 개 이상의 Core Domain에서 공통으로 필요한가.
3. 제품 전체의 일관성에 필요한가.
4. Runtime 또는 Resource 관리의 필수 기능인가.
5. 공식 Contract로 제공할 가치가 있는가.
6. Tenant, Permission 및 Policy 적용이 가능한가.
7. 확장 기능으로 분리하는 것보다 Core 포함이 타당한가.
8. 독립적 Lifecycle과 Version을 정의할 수 있는가.
9. 운영과 감사가 가능한가.
10. Product Vision과 Product Philosophy에 부합하는가.

조건을 충족하지 못하면 Extension 또는 Application Layer로 분리한다.

32. Core 금지사항

다음 구조는 허용하지 않는다.

- Tenant ID 없는 Core Resource

- Version 없는 Published Resource
- Lifecycle 검증 없는 상태 변경
- Policy 검증 없는 실행 요청
- Audit 없는 권한 변경
- Core Database에 대한 Extension 직접 접근
- Domain 간 Repository 직접 접근
- Resource Specification과 Status 혼합
- Published Resource 직접 수정
- Event 발행 없는 중요 상태 변경
- Idempotency 없는 실행 시작
- Schema 없는 Command와 Event
- 무제한 재시도
- 무기한 Lock
- Runtime Snapshot 없는 장기 실행
- Secret을 일반 Configuration에 저장
- 내부 오류 상세를 외부에 노출
- Read Model을 Source of Truth로 사용
- 특정 Provider SDK를 Domain Code에서 직접 사용
- Application 기능을 Core에 포함

33. Core Architecture 검증 기준

Core Component는 다음 기준을 충족해야 한다.

1. 명확한 Domain에 속하는가.
2. 책임이 하나로 정의되는가.
3. 공식 Interface가 존재하는가.

4. Tenant Context가 필수인가.
5. Identity와 Permission이 적용되는가.
6. Policy 평가 지점이 정의되는가.
7. Resource Version이 관리되는가.
8. Lifecycle이 정의되는가.
9. 입력과 출력 Schema가 존재하는가.
10. 상태 전이 규칙이 존재하는가.
11. Event가 정의되는가.
12. Audit 요구사항이 정의되는가.
13. Idempotency가 필요한지 판단되었는가.
14. 오류 계약이 정의되는가.
15. Retry와 Timeout이 정의되는가.
16. 관측성 정보가 제공되는가.
17. 장애가 다른 Domain으로 확산되지 않는가.
18. Provider 종속성이 Adapter로 분리되는가.
19. 테스트 가능한 구조인가.
20. Core에 포함될 타당성이 있는가.

34. Core Architecture의 최종 정의

AGEX Core는 AGEX의 모든 Resource, Version, Lifecycle, Permission, Policy, 실행 요청, 상태, Event, Audit 및 Usage를 일관된 방식으로 관리하는 공통 제어 영역이다.

AGEX Core는 Agent, Workflow, Knowledge, Memory, Plugin, Model 및 Application Layer의 구체적인 기능을 직접 구현하지 않는다.

대신 각 구성요소가 안전하고 예측 가능하며 재사용 가능한 방식으로 등록되고 연결되고 실행될 수 있도록 공통 계약과 제어 구조를 제공한다.

AGEX Core의 모든 Resource는 Tenant, Identity, Version, Lifecycle, Permission, Policy,

Schema 및 Audit 정보를 가진다.

모든 실행은 검증된 Resource Version과 Configuration Snapshot을 기준으로 생성되며, Runtime에 전달되기 전에 권한, 정책, 의존성, 비용 및 Quota 검증을 통과해야 한다.

모든 중요 상태 변경은 Event와 Audit Record를 생성하며, 확장 구성요소는 Core 내부에 직접 접근하지 않고 공식 API, Event 및 Extension Contract를 통해서만 연결된다.

AGEX Core Architecture는 다음 문장으로 최종 정의한다.

AGEX Core는 AI Operating System의 모든 자원과 실행을 표준화하고, 정책과 상태를 통제하며, 확장 가능한 공통 계약을 제공하는 제품의 중심 제어 구조이다.

본 문서는 AGEX Core의 구조, 책임, 경계 및 구현 원칙을 정의하는 공식 Core Architecture 문서로 사용한다.